Implementations for Image Restoration and Reconstruction

Generated by Grok 3

June 20, 2025

1 Introduction

This document provides separate Python implementations for each operation in Chapter 5 of *Digital Image Processing* (4th Edition, pp. 317368), covering image restoration and reconstruction techniques. Each implementation includes a function with a test case using a sample image.

2 Implementations

2.1 A Model of the Image Degradation/Restoration Process (p. 318)

```python
import numpy as np
import cv2
import matplotlib.pyplot as plt

# Simulate degradation model: g = h * f + n
def degrade_image(original, blur_sigma=2, noise_std=10):
    # Apply Gaussian blur as degradation function h
    blurred = cv2.GaussianBlur(original, (9, 9), blur_sigma)
    # Add Gaussian noise n
    noise = np.random.normal(0, noise_std, original.shape)
    degraded = np.clip(blurred + noise, 0, 255).astype(np.uint8)
    return degraded

# Test case
image = cv2.imread('lena.png', cv2.IMREAD_GRAYSCALE)
if image is None:
    raise FileNotFoundError("Image not found. Please provide 'lena.png'.")
degraded = degrade_image(image)
plt.imshow(degraded, cmap='gray')
plt.title('Degraded Image')
plt.show()
```

2.2 Noise Models (p. 318)

```
import numpy as np
import cv2
import matplotlib.pyplot as plt

# Add different noise models
def add_noise(image, noise_type='gaussian', noise_std=10,
   sp_ratio=0.05):
    if noise_type == 'gaussian':
        noise = np.random.normal(0, noise_std, image.shape)
        noisy = image + noise
    elif noise_type == 'salt_pepper':
        noisy = image.copy()
        num_salt = int(np.ceil(sp_ratio * image.size * 0.5))
        coords = [np.random.randint(0, i - 1, num_salt) for i in
           image.shape]
        noisy[coords[0], coords[1]] = 255
        num_pepper = int(np.ceil(sp_ratio * image.size * 0.5))
        coords = [np.random.randint(0, i - 1, num_pepper) for i
           in image.shape]
        noisy[coords[0], coords[1]] = 0
    return np.clip(noisy, 0, 255).astype(np.uint8)

# Test case
image = cv2.imread('lena.png', cv2.IMREAD_GRAYSCALE)
if image is None:
    raise FileNotFoundError("Image not found. Please provide
       'lena.png'.")
gaussian_noisy = add_noise(image, 'gaussian', noise_std=20)
sp_noisy = add_noise(image, 'salt_pepper', sp_ratio=0.05)
plt.subplot(121), plt.imshow(gaussian_noisy, cmap='gray'),
   plt.title('Gaussian Noise')
plt.subplot(122), plt.imshow(sp_noisy, cmap='gray'),
   plt.title('Salt-and-Pepper Noise')
plt.show()
```

2.3 Restoration in the Presence of Noise OnlySpatial Filtering (p. 327)

```
import cv2
import matplotlib.pyplot as plt

# Apply spatial filtering
def spatial_filtering(image):
    mean_filtered = cv2.blur(image, (5, 5))
    median_filtered = cv2.medianBlur(image, 5)
    return mean_filtered, median_filtered

# Test case
```

```
image = cv2.imread('lena.png', cv2.IMREAD_GRAYSCALE)
if image is None:
    raise FileNotFoundError("Image not found. Please provide 'lena.png'.")
noisy = add_noise(image, 'gaussian', noise_std=20)  # Assuming add_noise from previous section
mean_filtered, median_filtered = spatial_filtering(noisy)
plt.subplot(121), plt.imshow(mean_filtered, cmap='gray'), plt.title('Mean Filter')
plt.subplot(122), plt.imshow(median_filtered, cmap='gray'), plt.title('Median Filter')
plt.show()
```

2.4 Periodic Noise Reduction Using Frequency Domain Filtering (p. 340)

```
import numpy as np
import cv2
import matplotlib.pyplot as plt

# Add periodic noise and reduce it
def add_periodic_noise(image, amplitude=50, freq=0.1):
    x, y = np.meshgrid(np.arange(image.shape[1]), np.arange(image.shape[0]))
    noise = amplitude * np.sin(2 * np.pi * freq * x)
    return np.clip(image + noise, 0, 255).astype(np.uint8)

def periodic_noise_reduction(image):
    f = np.fft.fft2(image)
    fshift = np.fft.fftshift(f)
    rows, cols = image.shape
    crow, ccol = rows // 2, cols // 2
    notch = np.ones((rows, cols))
    notch[crow-10:crow+10, ccol-10:ccol+10] = 0
    fshift_filtered = fshift * notch
    f_ishift = np.fft.ifftshift(fshift_filtered)
    img_back = np.fft.ifft2(f_ishift).real
    return np.clip(img_back, 0, 255).astype(np.uint8)

# Test case
image = cv2.imread('lena.png', cv2.IMREAD_GRAYSCALE)
if image is None:
    raise FileNotFoundError("Image not found. Please provide 'lena.png'.")
periodic_noisy = add_periodic_noise(image)
filtered = periodic_noise_reduction(periodic_noisy)
plt.imshow(filtered, cmap='gray')
plt.title('Periodic Noise Reduced')
plt.show()
```

2.5 Linear, Position-Invariant Degradations (p. 348)

```
import cv2
import matplotlib.pyplot as plt

# Simulate linear, position-invariant degradation
def apply_degradation(image, psf_size=9, sigma=2):
    psf = cv2.getGaussianKernel(psf_size, sigma)
    psf = np.outer(psf, psf)
    degraded = cv2.filter2D(image, -1, psf)
    return np.clip(degraded, 0, 255).astype(np.uint8)

# Test case
image = cv2.imread('lena.png', cv2.IMREAD_GRAYSCALE)
if image is None:
    raise FileNotFoundError("Image not found. Please provide
        'lena.png'.")
degraded = apply_degradation(image)
plt.imshow(degraded, cmap='gray')
plt.title('Linear Degradation')
plt.show()
```

2.6 Estimating the Degradation Function (p. 352)

```
import cv2
import matplotlib.pyplot as plt

# Simplified estimation (assuming known blur)
def estimate_degradation(image, degraded):
    psf_size = 9
    sigma = 2
    psf = cv2.getGaussianKernel(psf_size, sigma)
    psf = np.outer(psf, psf)
    return psf

# Test case
image = cv2.imread('lena.png', cv2.IMREAD_GRAYSCALE)
if image is None:
    raise FileNotFoundError("Image not found. Please provide
        'lena.png'.")
degraded = apply_degradation(image)  # Assuming
    apply_degradation from previous section
estimated_psf = estimate_degradation(image, degraded)
plt.imshow(estimated_psf, cmap='gray')
plt.title('Estimated PSF')
plt.show()
```

2.7 Inverse Filtering (p. 356)

```python
import numpy as np
import cv2
import matplotlib.pyplot as plt

# Inverse filtering
def inverse_filter(degraded, psf):
    degraded_freq = np.fft.fft2(degraded)
    psf_freq = np.fft.fft2(psf, s=degraded.shape)
    restored_freq = degraded_freq / (psf_freq + 1e-10)  # Avoid
       division by zero
    restored = np.fft.ifft2(restored_freq).real
    return np.clip(restored, 0, 255).astype(np.uint8)

# Test case
image = cv2.imread('lena.png', cv2.IMREAD_GRAYSCALE)
if image is None:
    raise FileNotFoundError("Image not found. Please provide
       'lena.png'.")
degraded = apply_degradation(image)  # Assuming apply_degradation
psf = cv2.getGaussianKernel(9, 2)
psf = np.outer(psf, psf)
restored = inverse_filter(degraded, psf)
plt.imshow(restored, cmap='gray')
plt.title('Inverse Filtered')
plt.show()
```

2.8 Minimum Mean Square Error (Wiener) Filtering (p. 358)

```python
import numpy as np
import cv2
import matplotlib.pyplot as plt

# Wiener filtering
def wiener_filter(degraded, psf, k=0.01):
    degraded_freq = np.fft.fft2(degraded)
    psf_freq = np.fft.fft2(psf, s=degraded.shape)
    wiener_freq = np.conj(psf_freq) / (np.abs(psf_freq)**2 + k)
    restored_freq = degraded_freq * wiener_freq
    restored = np.fft.ifft2(restored_freq).real
    return np.clip(restored, 0, 255).astype(np.uint8)

# Test case
image = cv2.imread('lena.png', cv2.IMREAD_GRAYSCALE)
if image is None:
    raise FileNotFoundError("Image not found. Please provide
       'lena.png'.")
degraded = apply_degradation(image)  # Assuming apply_degradation
psf = cv2.getGaussianKernel(9, 2)
psf = np.outer(psf, psf)
```

```
restored = wiener_filter(degraded, psf)
plt.imshow(restored, cmap='gray')
plt.title('Wiener Filtered')
plt.show()
```

2.9 Constrained Least Squares Filtering (p. 363)

```
import numpy as np
import cv2
import matplotlib.pyplot as plt

# Simplified constrained least squares (basic regularization)
def constrained_least_squares(degraded, psf, lambda_=0.01):
    degraded_freq = np.fft.fft2(degraded)
    psf_freq = np.fft.fft2(psf, s=degraded.shape)
    # Simple regularization (actual implementation requires Laplacian constraint)
    restored_freq = degraded_freq * np.conj(psf_freq) / (np.abs(psf_freq)**2 + lambda_)
    restored = np.fft.ifft2(restored_freq).real
    return np.clip(restored, 0, 255).astype(np.uint8)

# Test case
image = cv2.imread('lena.png', cv2.IMREAD_GRAYSCALE)
if image is None:
    raise FileNotFoundError("Image not found. Please provide 'lena.png'.")
degraded = apply_degradation(image)  # Assuming apply_degradation
psf = cv2.getGaussianKernel(9, 2)
psf = np.outer(psf, psf)
restored = constrained_least_squares(degraded, psf)
plt.imshow(restored, cmap='gray')
plt.title('Constrained Least Squares')
plt.show()
```

2.10 Geometric Mean Filter (p. 367)

```
import numpy as np
import cv2
import matplotlib.pyplot as plt

# Geometric mean filter (simplified)
def geometric_mean_filter(degraded, psf, k=0.01):
    degraded_freq = np.fft.fft2(degraded)
    psf_freq = np.fft.fft2(psf, s=degraded.shape)
    geo_freq = (np.conj(psf_freq) / (np.abs(psf_freq)**2 + k)) ** 0.5  # Geometric mean approach
    restored_freq = degraded_freq * geo_freq
    restored = np.fft.ifft2(restored_freq).real
```

```
    return np.clip(restored, 0, 255).astype(np.uint8)

# Test case
image = cv2.imread('lena.png', cv2.IMREAD_GRAYSCALE)
if image is None:
    raise FileNotFoundError("Image not found. Please provide 'lena.png'.")
degraded = apply_degradation(image)  # Assuming apply_degradation
psf = cv2.getGaussianKernel(9, 2)
psf = np.outer(psf, psf)
restored = geometric_mean_filter(degraded, psf)
plt.imshow(restored, cmap='gray')
plt.title('Geometric Mean Filter')
plt.show()
```

2.11 Image Reconstruction from Projections (p. 368)

```
from skimage.transform import radon, iradon
import cv2
import matplotlib.pyplot as plt

# Image reconstruction from projections
def reconstruct_image(image):
    theta = np.linspace(0., 180., 180, endpoint=False)
    sinogram = radon(image, theta=theta)
    reconstructed = iradon(sinogram, theta=theta, filter_name='ramp')
    return np.clip(reconstructed, 0, 255).astype(np.uint8)

# Test case
image = cv2.imread('lena.png', cv2.IMREAD_GRAYSCALE)
if image is None:
    raise FileNotFoundError("Image not found. Please provide 'lena.png'.")
reconstructed = reconstruct_image(image)
plt.imshow(reconstructed, cmap='gray')
plt.title('Reconstructed Image')
plt.show()
```

3 Notes

- **Dependencies**: Install `numpy`, `opencv-python`, `scikit-image`, and `matplotlib` using `pip`.
- **Image**: Requires a grayscale image `lena.png` (512x512 recommended). Place it in the working directory.
- **Limitations**: - Inverse filtering may amplify noise due to division by small values. - Constrained Least Squares and Geometric Mean Filter are simplified; full

implementations require additional constraints (e.g., Laplacian). - Reconstruction assumes a full image; real-world CT data would require projection data.

- **Usage**: Run each code block independently after installing dependencies and providing the image.

4 References

References

[1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018, pp. 317368.

[2] Harris, C.R., et al. "Array programming with NumPy." *Nature*, vol. 585, no. 7825, 2020, pp. 357362. `https://numpy.org`

[3] Bradski, G. "The OpenCV Library." *Dr. Dobb's Journal of Software Tools*, 2000. `https://opencv.org`

[4] van der Walt, S., et al. "scikit-image: Image processing in Python." *PeerJ*, vol. 2, 2014, e453. `https://scikit-image.org`

[5] Hunter, J.D. "Matplotlib: A 2D graphics environment." *Computing in Science & Engineering*, vol. 9, no. 3, 2007, pp. 9095. `https://matplotlib.org`